Practical:05 Date:18-09-2026

Gradient Decent and XGBoost

Write a python program to predict a non-linear target variable y using two predictors x_1 and x_2.

$Y = 3x_1^2+2\sin(x_2) + €$

Where $€ \sim N(0, 2^2)$ is a random noise.

· Implement Multiple Linear Regressions using Gradient Descent (not using built-in libraries like LinearRegression).

· Compare its performance with a nonlinear machine learning model (XGBoost) on the same dataset.

Procedure:

1. Import the required libraries for numerical calculations, plotting, data splitting, scaling, evaluation, and XGBoost.
2. Generate the dataset by creating two input variables, x1 and x2, with 500 random observations.
3. Create the target variable y using a nonlinear equation involving x12x_1^2 and sin(x2)\sin(x_2), along with random noise.
4. Prepare the input matrix X using x1 and x2 as predictor variables.
5. Split the data into training data (70%) and testing data (30%).
6. Standardize the predictor variables using StandardScaler so that the variables have a common scale.
7. Add an intercept term to the standardized training and testing data for linear regression.
8. Initialize the regression weights and set the learning rate and number of epochs for Gradient Descent.
9. Train the Linear Regression model using Gradient Descent by:
    - Calculating predictions.
    - Calculating the gradient.
    - Updating the weights.
    - Calculating and storing the loss for each epoch.
10. Make predictions on the test data using the trained Linear Regression model.
11. Evaluate Linear Regression using Mean Squared Error (MSE) and R^2 score.

12. Calculate the regression coefficients and intercept in the original scale of the variables.
13. Create an XGBoost Regression model with specified parameters such as number of trees, learning rate, tree depth, and regularization.
14. Train the XGBoost model using the training data.
15. Predict the target values for the test data using XGBoost.
16. Evaluate XGBoost using MSE and R^2 score.
17. Display the results of both Linear Regression and XGBoost.
18. Plot the Gradient Descent loss curve to show how the loss changes across epochs.
19. Create actual vs. predicted scatter plots for both Linear Regression and XGBoost.
20. Compare the two models based on their MSE and R^2 values and visualize their prediction performance.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score
from xgboost import XGBRegressor

np.random.seed(42)
n = 500
x1 = np.random.uniform(-5, 5, n)
x2 = np.random.uniform(-5, 5, n)

y = 3 * (x1 ** 2) + 2 * np.sin(x2) + np.random.normal(0, 2, n)
X = np.vstack((x1, x2)).T

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.3,random_state=42)
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

Xb_train = np.c_[np.ones(X_train_scaled.shape[0]),X_train_scaled]
Xb_test = np.c_[np.ones(X_test_scaled.shape[0]),X_test_scaled]

w = np.zeros(Xb_train.shape[1])
alpha = 0.05
epochs = 1000
m = len(y_train)
losses = []
for epoch in range(epochs):
    #Prediction
    y_pred = np.dot(Xb_train, w)
    #Gradient
    grad = (1 / m) * np.dot(
        Xb_train.T,
        (y_pred - y_train))
    #Update weights
    w -= alpha * grad
    #loss
    loss = (1 / (2 * m)) * np.sum(
        (y_pred - y_train) ** 2)
    losses.append(loss)

y_pred_lr = np.dot(Xb_test, w)
mse_lr = mean_squared_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

beta1 = w[1] / scaler.scale_[0]
```

```
beta2 = w[2] / scaler.scale_[1]
intercept = (w[0]- beta1 * scaler.mean_[0]- beta2 * scaler.mean_[1])

xgb_model = XGBRegressor(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=4,
    reg_lambda=1,
    random_state=42)
xgb_model.fit(X_train, y_train)

y_pred_xgb = xgb_model.predict(X_test)
mse_xgb = mean_squared_error(y_test,y_pred_xgb)
r2_xgb = r2_score(y_test,y_pred_xgb)

print("Linear Regression (Gradient Descent):")
print(f"Equation: y = {intercept:.3f} "f"+ ({beta1:.3f})*x1 "f"+ ({beta2:.3f})*x2")
print(f"Test MSE: {mse_lr:.4f}, "f"R²: {r2_lr:.4f}\n")
print("XGBoost:")
print(f"Test MSE: {mse_xgb:.4f}, "f"R²: {r2_xgb:.4f}")

plt.figure(figsize=(12, 4))
plt.plot(losses)
plt.title("Gradient Descent Loss Curve (Linear Regression)")
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.show()

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
```

```
plt.scatter(y_test,y_pred_lr,alpha=0.6,color="red")
plt.plot([y_test.min(), y_test.max()],[y_test.min(), y_test.max()],"k--")

plt.title("Linear Regression (Gradient Descent)")
plt.xlabel("Actual y")
plt.ylabel("Predicted y")

# XGBoost
plt.subplot(1, 2, 2)
plt.scatter(y_test,y_pred_xgb,alpha=0.6,color="green")
plt.plot([y_test.min(), y_test.max()],[y_test.min(), y_test.max()],"k--")
plt.title("XGBoost Predictions")
plt.xlabel("Actual y")
plt.ylabel("Predicted y")
plt.tight_layout()
plt.show()
```

Output:

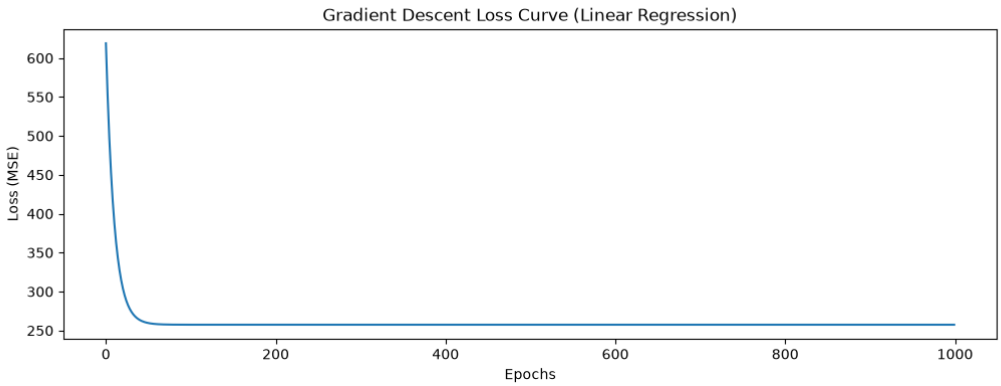
Linear Regression (Gradient Descent):

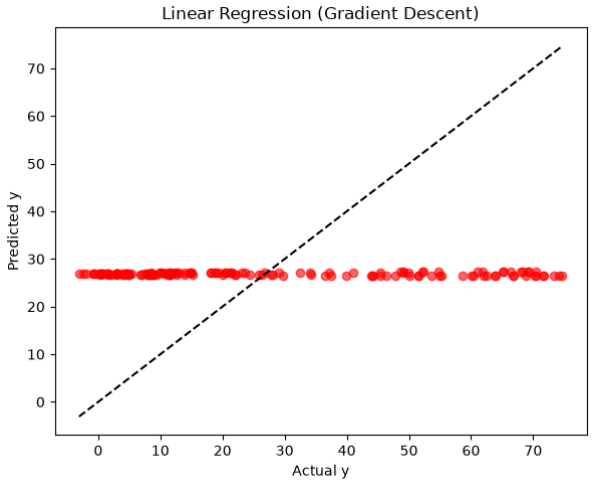
Equation: y = 26.881 + (0.085)*x1 + (-0.003)*x2

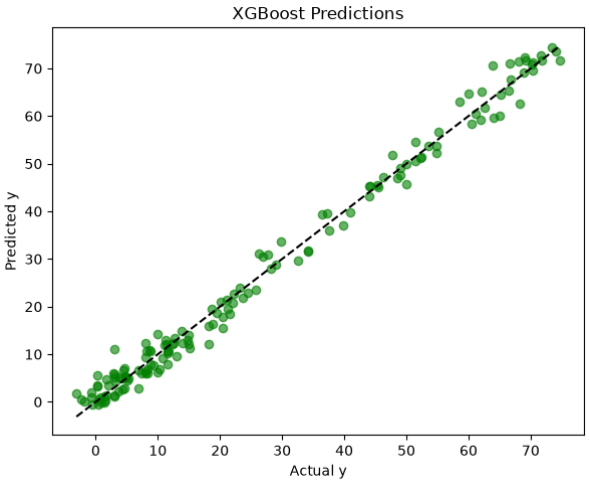
Test MSE: 589.3888, R^2: -0.0037

XGBoost:

Test MSE: 6.2443, R^2: 0.9894


Gradient Descent Loss Curve (Linear Regression)
Loss (MSE)
Epochs
600
550
500
450
400
350
300
250
0
200
400
600
800
1000


Linear Regression (Gradient Descent)
Predicted y
Actual y
70
60
50
40
30
20
10
0
0
10
20
30
40
50
60
70


XGBoost Predictions
Predicted y
Actual y
70
60
50
40
30
20
10
0
0
10
20
30
40
50
60
70